WPT Day 15 Morning

Abhishek Purohit · Wed 04 Nov 26 · Create-product form with MUI `TextField`, the 12-column Grid system, typed form state with a generic `onChange` and `onSubmit`, Axios `POST` through the existing API client, React Toastify wrapped in `src/utils`, and product-list delete with `IconButton`, confirmation `Dialog` and `DELETE /product/:id`

Table of Contents

1. MUI TextField and the Create Product Form
2. Grid System
3. Form State
4. Axios POST
5. Toast Utils
6. Delete Dialog
7. Homework, Validation and What Comes Next
8. Combined Worked Example
9. Quick Revision Checklist
10. Practice Exercises
11. Practice Answers

MUI TextField and the Create Product Form

Problem: the product page still cannot take data

The product catalogue already has a backend (`POST /product`, `GET /product`, `DELETE /product/:id` from the Express + MySQL work) and a list screen. What it does **not** yet have is a form that a user can fill and submit from the UI. Until today that insert was done in **Postman**. The job of this session is to build that form in the existing `CreateProduct` page and then wire it to the same API.

The form needs **four fields** plus a submit button:

Field	Label on screen	`name` attribute used in class
Product name	Name of the product	`name`
Description	Description	`description`
Price	Price	`price`
Quantity	Quantity	`qty`

Validations (required, no negative price or quantity) are **deliberately left for later** — the lecturer flagged React Hook Form for that. Today the form only has to collect values, send them, and report success or failure.

Mechanism: MUI `TextField` instead of a raw `<input>`

MUI's `TextField` is a styled input. You give it a `label`, a `name` (so the form handler can identify the field), and a `variant` that decides how it looks.

`variant`	What you see
`outlined`	Box with a border all around (the one used in class)
`filled`	Grey filled body, lighter underline feel
`standard`	Underline only, no border box

`id` is optional for this form; the lecturer left it off.

Two ways to wrap the fields in a real `<form>`

The heading is an MUI `Alert` that reads **Add a New Product**, sitting inside an MUI `Box` with `margin-top: 30px` so it is not glued to the top of the page.

Under that heading you need a `<form>`, because submit (and `onSubmit`) belongs to a form, not to a loose pile of inputs. Two legal approaches:

Approach	What you write	What you get
Native HTML	`<form onSubmit={handleSubmit}>`	A real form. No extra MUI `Box` behaviour.
MUI polymorphic `Box`	`<Box component="form" onSubmit={handleSubmit}>`	MUI still renders a `<form>` **behind the scenes**, but you keep `Box` spacing, `sx`, and the MUI layout model.

The lecturer's phrasing: *on behalf of this Box, your form component is now going to be rendered.* Choice is yours. Class used the `Box component="form"` path so the form can still take MUI layout props.

Background: MUI's `component` prop is the polymorphic trick: `Box` is a styled `div` by default; `component="form"` changes the underlying HTML element without losing the styling API.

Stack: a first, non-responsive layout

Pasting four `TextField`s one after another makes each one sit on its own line and stretch to the **full browser width**, with **no gap** between them. That is readable but ugly.

MUI `Stack` stacks children one after another (a column by default). The `spacing` prop is the **vertical gap** between those children.

```
<Stack spacing={2}>
  <TextField label="Name of the product" name="name" variant="outlined" />
  <TextField label="Description" name="description" variant="outlined" />
  <TextField label="Price" name="price" variant="outlined" />
  <TextField label="Quantity" name="qty" variant="outlined" />
</Stack>
```

The lecturer then put `width: 300` on the outer `Box` so the fields stopped eating the whole screen. That is the next problem.

Lecture's first sketch (before Grid)

```
import { Alert, Box, TextField, Stack } from "@mui/material";

export function CreateProduct() {
  return (
    <Box sx={{ mt: "30px", width: 300 }}>
      <Alert severity="info">Add a New Product</Alert>
      <Box component="form">
        <Stack spacing={2}>
          <TextField label="Name of the product" name="name" variant="outlined" />
          <TextField label="Description" name="description" variant="outlined" />
          <TextField label="Price" name="price" variant="outlined" />
          <TextField label="Quantity" name="qty" variant="outlined" />
        </Stack>
      </Box>
    </Box>
  );
}
```

> **Pitfall:** A hardcoded `width: 300` occupies **300 pixels on a phone, a tablet and a laptop**. The form does not grow or shrink with the screen. That is why the lecture abandoned `Stack` + a fixed width and moved to the Grid system.

> **Exam:** `TextField` `variant` is `outlined` | `filled` | `standard`. A form can be a raw `<form>` or `<Box component="form">`. `Stack` + `spacing` lines fields up with a gap; it is not a responsive 12-column layout.

Grid System

Problem: one width cannot serve every device

`Stack` solved "fields on new lines with a gap". It did **not** solve "this form should look right on a phone and on a wide monitor". A 300 px box is too narrow on a desktop and still not truly responsive — it is just a smaller fixed box.

Mechanism: break the browser into 12 equal columns

This is the same design principle used by **Bootstrap, MUI, Daisy UI and every mainstream responsive UI library**:

1. The available width is divided into **12 equal columns**.
2. Each section of the page is told **how many of those 12 columns it occupies**.
3. On a wide screen, sections that share one row sit **side by side**.
4. When the viewport shrinks (resize, tablet, phone) and there is not enough width, those sections **wrap onto a new line** so the UI does not break.

The lecturer's rule: you may invent any layout you like, but **the column counts in one row should add up to 12**. Then MUI can keep the row responsive.

Partition examples from class

```
Browser width = 12 equal columns

 [1][2][3][4][5][6][7][8][9][10][11][12]

6 + 6
 [###### first (6) ######][##### second (6) #####]

4 + 4 + 4
 [## A (4) ##][## B (4) ##][## C (4) ##]

8 + 4
 [########## wide (8) ##########][## side (4) ##]

2 + 8 + 2
 [2][########## main (8) ##########][2]
```

On a large screen a `6 + 6` row is two fields side by side. Shrink the window and those two blocks drop **one under the other** so nothing overflows.

> **Exam:** Responsive UI libraries partition the viewport into **12 columns**. A row's sections should sum to 12 (`6+6` , `4+4+4` , `8+4` , `2+8+2`). On a small screen those sections wrap to a new line instead of breaking the layout.

Grid container is the row; Grid without `container` is the column

MUI does **not** ship components named `Row` and `Column` . There is one component, `Grid` , and a prop that changes its role:

What you want	How you write it	Role
A row	`<Grid container>`	The parent that owns the 12-column track
A column	`<Grid size={6}>` (no `container`)	A cell that occupies `size` columns of that row

The lecturer: *if you want to create a row, denote that Grid as a container; the columns are Grid components without this container prop.*

Background: Older MUI Grid (v5) used `<Grid item xs={6}>` . The lecture's wording is `size={6}` , which is MUI Grid v2 / MUI v6. The idea is identical: "this cell takes 6 of the 12 columns." In DevTools the lecture also pointed at a Bootstrap-like `col-lg-6` class on the generated markup — that is the same 6-of-12 idea, not a different system.

The layout built in class

Two rows of two fields, then a submit row:

```
Row 1 (container):  [ Name  size=6 ]  [ Description  size=6 ]
Row 2 (container):  [ Price size=6 ]  [ Quantity     size=6 ]
Row 3:              [ Add Product button ]
```

Each field lives **inside** its column Grid. The first `Stack` is commented out once this is in place. The hardcoded `width: 300` on the outer Box is removed so the two `size={6}` columns can actually share the browser width.

`fullWidth` : the field fills its column, not the whole browser

A `TextField` has a default, fairly small intrinsic width. Sitting inside a `size={6}` column, it does **not** automatically grow to that column.

`fullWidth` makes the input occupy **100% of its parent**. The parent here is the column Grid (half the row), **not** the whole window. After both name and description get `fullWidth`, they sit as two equal, wide fields.

```
Without fullWidth
  [ short input ]          [ short input ]
  [—— size 6 ——]           [—— size 6 ——]

With fullWidth
  [═══════ name ═══════]   [═══════ desc ═══════]
  [—— size 6 ——]           [—— size 6 ——]
```

> **Pitfall:** `fullWidth` does not mean "full browser". It means "full width of the Grid cell that wraps this field."

Horizontal gap: `spacing` on the container

Two `size={6}` columns with `fullWidth` fields sit **flush** against each other. The gap belongs on the **row** (`Grid container`), not on the field:

```
<Grid container spacing={3}>
  <Grid size={6}> … </Grid>
  <Grid size={6}> … </Grid>
</Grid>
```

Class tried `spacing={3}` , then `spacing={5}` . That number is the MUI spacing scale (theme spacing units), not "3% of the screen" in CSS terms — the lecturer said "3% of space" loosely while turning the knob.

Vertical gap between two containers: `rowSpacing` did not help

There are now **two** `Grid container` s (row 1 and row 2). They are siblings, not rows inside one parent grid. `rowSpacing` / `columnSpacing` apply **inside one container** — they space that container's own children. They do not insert a gap **between two separate containers**.

That is why the live experiment (`rowSpacing={2}` , then `4` , then `5` on each container) showed **no vertical gap** between the name/description row and the price/quantity row.

Shivam Soni's observation in class: the gap would appear if both rows lived inside **one main container**. That is the correct Grid reading.

What the lecture actually shipped: they stopped fighting `rowSpacing` and put `sx={{ mb: 4 }}` (margin-bottom) on the first container. Same visual result, local workaround.

```jsx
<Grid container spacing={5} sx={{ mb: 4 }}>
  <Grid size={6}>
    <TextField fullWidth label="Name of the product" name="name" variant="outlined" />
  </Grid>
  <Grid size={6}>
    <TextField fullWidth label="Description" name="description" variant="outlined" />
  </Grid>
</Grid>

<Grid container spacing={5}>
  <Grid size={6}>
    <TextField fullWidth label="Price" name="price" variant="outlined" />
  </Grid>
  <Grid size={6}>
    <TextField fullWidth label="Quantity" name="qty" variant="outlined" />
  </Grid>
</Grid>
```

Pitfall: `rowSpacing` on two independent `Grid container`s will not space those two rows apart. Either wrap both rows in one parent `Grid container` (the cleaner model) or add margin (`sx={{ mb: 4 }}`) on the first row.

Submit button

MUI `Button` , `type="submit"` so it fires the form's `onSubmit` , label **Add Product**.

Button look is `variant` :

`variant`	**Look**
`text`	Label only, no fill or outline
`contained`	Filled (the one used; the lecturer said "content" — he meant **contained**)
`outlined`	Border, transparent fill

You can also set `size` and `color` (`primary` , `success` , `error` , ...).

```jsx
<Button type="submit" variant="contained">
  Add Product
</Button>
```

Exam-ready takeaway

Idea	Fact
12-column grid	Viewport split into 12 equal columns; a row should sum to 12
`container`	This Grid is the row
no `container` + `size`	This Grid is a column
`fullWidth`	Input fills its **column**
`spacing`	Gap between cells **inside** that container
Why not 300 px	Same pixel width on every device; not responsive

---

Form State

Problem: the fields are still dead

A pretty form that does not store what the user types is a picture. Day 13 already fixed this for a registration page: **one state object**, a **generic** `onChange`, then `onSubmit` + `preventDefault`. Today that pattern is applied to the product form, with TypeScript types on the state and on the event objects.

Mechanism: one `formData` object

```
const [formData, setFormData] = useState<IFormData>({
  name: "",
  description: "",
  price: "",
  qty: "",
});
```

Keys match the `name` attributes. First sketch in class used `price: 0` and `qty: 0`; they switched to **blank strings** so the boxes start empty instead of showing `0`.

`IFormData` in `types.ts`

A dedicated `types.ts` (or similar) exports the shape. Class used an interface named `IFormData` / product form data:

```ts
export interface IFormData {
  name: string;
  description: string;
  price: number | string;
  qty: number | string;
}
```

Number vs string for price and quantity

This was a **deliberate design choice**, not a TypeScript trivia item:

If you need…	Store price / qty as…
Arithmetic on the client (`price * qty` , discounts, totals)	`number`
Only to collect the value and **send it to the server**	`string` is fine — `event.target.value` is always a string anyway

Lecturer: *if there is no arithmetic operation, you can go with the string data type also. Choice is yours.* Class ended on blank strings.

> **Exam:** `event.target.value` is a string. Use `number` for a field only when the **frontend** will do arithmetic on it. Otherwise a string that travels in the JSON body is enough.

`handleChange` : one function, dynamic key

Every field gets the **same** `onChange={handleChange}` . React passes a change event. From React you import `ChangeEvent` and specialise it to an input:

```ts
import { ChangeEvent } from "react";

function handleChange(event: ChangeEvent<HTMLInputElement>) {
  setFormData({
     ... formData,
    [event.target.name]: event.target.value,
  });
}
```

Piece	Job
`... formData`	Spread: copy the keys you are **not** editing, or they disappear
`[event.target.name]`	Computed property: the input's `name` becomes the state key

Piece	Job
`event.target.value`	Whatever the user just typed

That is why `name="qty"` is not cosmetic. If the attribute were `quantity` but the state key were `qty`, typing in Quantity would write a **new** key and leave `qty` empty.

> **Pitfall:** `setFormData({ name: event.target.value })` wipes `description`, `price` and `qty`. Always spread first. And the `name` attribute must equal the state key (`qty` / `qty`, not `quantity` / `qty`).

`handleSubmit`: the form's event, not the button's

`onSubmit` goes on the **form** (`Box component="form"`), not on the button. The button is only `type="submit"`.

Class imported `SubmitEvent` from `react` and typed the argument as a submit event of an HTML form element. In many `@types/react` versions the usual name is `FormEvent<HTMLFormElement>` — same idea: this is the form-submit event, not a click event.

```
import { FormEvent } from "react";

async function handleSubmit(event: FormEvent<HTMLFormElement>) {
  event.preventDefault(); // stop the browser reload
  console.log(formData);  // first checkpoint, before the API
}
```

`preventDefault()` is mandatory. Without it the browser posts the form as a real navigation, the page reloads, React state dies, and you never see the Axios call finish.

First live check in class (console only, no API yet):

Field	Value typed
name	`mouse`
description	`Bluetooth mouse`
price	`456`
qty	`10`

The console showed the whole object. Only then did they attach Axios.

Exam-ready takeaway

The Day 13 three-step form is unchanged:

1. **One state object** whose keys match `name` attributes.
2. **One** `onChange` that spreads + `[event.target.name]`.
3. `onSubmit` + `preventDefault` on the form.

TypeScript only annotates what you already knew: `useState<IFormData>`, `ChangeEvent<HTMLInputElement>`, form submit event.

Axios POST

Problem: the object is sitting in React state, the database does not know

Postman used to send `POST /product` with a JSON body. The lecturer's line: *I have told you one day will come when we do this programmatically from our frontend, and today that day has come.*

```
[ CreateProduct form ]
          |
          |  formData  { name, description, price, qty }
          v
[ apiClient.post("/product", formData) ]
          |
          v
[ Express POST /product ]  ⟶  INSERT into product
          |
          v
[ JSON response ]  e.g. { message: "product inserted", insertedId: 4 }
```

The client never talks to MySQL. The frontend talks to the API; the API talks to the database. Same three-actor line as Day 8.

Mechanism: the existing Axios instance, not a raw `axios.post`

They already created an Axios **instance** (`apiClient`) with `baseURL` and `timeout`. That instance already knows the host. The call only names the **sub-URL** and the body:

```
const response = await apiClient.post("/product", formData);
```

Argument	Meaning
`"/product"`	Path appended to the configured `baseURL`

Argument	Meaning
`formData`	Request body (Axios JSON-serialises the object)

`post` returns a Promise, so the handler is `async` and the call is `await`ed. The result is a full Axios response: `status`, `headers`, `data`, timing. The payload the **API author** sent is `response.data`.

Live insert in class (after empty-cache reload):

Field	Value
name	`mouse`
description	`wireless mouse`
price	`654`
qty	`13`

`response.data` came back as **product inserted** with **inserted ID 4**. That is the confirmation that the UI has done Postman's job.

`try / catch` so a failed API does not crash the page

```
try {
  const response = await apiClient.post("/product", formData);
  console.log(response);
  console.log(response.data);
} catch (error) {
  console.log(error);
  // later: toast.error( ... )
}
```

A 500 from Express rejects the Axios promise and lands in `catch`. Class first only logged it; the toast section turns both branches into UI.

Background: Axios treats HTTP status >= 400 as a thrown error. `response.data` on success is the API JSON; in `catch` you typically read `error.response.data` if you need the server's error body.

> **Exam:** Frontend save = `await apiClient.post("/product", formData)`. The instance already has `baseURL`. The useful payload is `response.data`. Wrap the call in `try/catch` so a 500 does not white-screen the app.

> **Pitfall:** The JSON keys you send are exactly the state keys. Day 8's insert SQL read `quantity`. This form's key is `qty`. If the backend still destructures `quantity`, the column will be `NULL` / `undefined` even though the UI showed a number. Align the names or map them before POST.

Toast Utils

Problem: the database changed and the screen stayed silent

After the successful insert, the only confirmation was the **console**. A real user never opens DevTools. The lecturer: *we haven't got any confirmation onto the UI.*

Mechanism: React Toastify

React Toastify is the library used in class (`npm i react-toastify` after stopping the dev server). Two pieces must exist:

1. `<ToastContainer />` **once**, in `App.tsx`, so every page can raise a toast.
2. A **function call** (`toast.success` , `toast.error` , `toast.info` , ...) at the moment you know the outcome.

You do **not** mount a new container inside `CreateProduct`.

Demo settings they clicked through on the docs site:

Knob	Values shown
Position	top-right (chosen)
Type	info, success, error
Theme	light, dark, colored
Extra	auto-close timer, progress bar, pause on hover so the user can finish reading, close on click, newest-on-top

Transitions they previewed: **slide**, **flip**, **zoom**, **bounce**. Class copied a **top-right / colored / success / Bounce** config into the success call.

```
import { toast, Bounce } from "react-toastify";

toast.success("Product created", {
```

```
  position: "top-right",
  autoClose: 5000,
  hideProgressBar: false,
  closeOnClick: true,
  pauseOnHover: true,
  draggable: true,
  theme: "colored",
  transition: Bounce,
});
```

In `catch`, the same config with `toast.error("Product creation failed")` (or "something went wrong").

Live success: **Data cable** / **Type C2C** / **765** / **17** → toast "Product created".

Live failure: they **renamed the product table** in the SQL `INSERT` so the query exploded, Express returned **500**, Axios threw, `catch` ran, toast showed **Product Creation Failed**. Then they put the table name back. The point: **success and failure both stay on a living page**. The app does not crash.

Why this config must not be copy-pasted into 50 components

Two problems, both said out loud:

1. **DRY.** Fifty components would each carry the same position / theme / Bounce blob. The message changes; the config does not.
2. **Library lock-in.** *If tomorrow at 8 pm the React Toastify owner says the project is shutting down, your Lanka will burn* — because every component imported `react-toastify` directly. Swapping the library would mean touching every file.

Mechanism: `src/utils/toastUtil.ts`

This is **not** a component and **not** an API helper. It is a **utility**: a function used from many pages. Class created `src/utils/` for exactly that kind of code (toasts, date conversion, small calculations).

Name the file after the **job**, not the library: `toastUtil.ts`, **not** `reactToastify.ts`. Tomorrow the body can call a different package; the rest of the app still writes `showSuccessToast("Product created")`.

```
import { toast, Bounce, type ToastOptions } from "react-toastify";

const toastConfig: ToastOptions = {
  position: "top-right",
  autoClose: 5000,
  hideProgressBar: false,
```

```
  closeOnClick: true,
  pauseOnHover: true,
  draggable: true,
  theme: "colored",
  transition: Bounce,
};

export function showSuccessToast(message: string) {
  toast.success(message, toastConfig);
}

export function showErrorToast(message: string) {
  toast.error(message, toastConfig);
}
```

Argument name is `message`, not `errorMessage` — both helpers take a generic string.

`CreateProduct` then becomes:

```
showSuccessToast("Product created");
// ...
showErrorToast("Product creation failed");
```

Live check after the extract: **SSD** / **SSD from Seagate** / **4567** / **12** → same success toast, now via the util.

The TypeScript fight they walked through and then abandoned

A raw object literal for `position: "top-right"` is inferred as `string`. Toastify's `position` is a **union of fixed strings** (`top-right` | `top-center` | `top-left` | `bottom-right` | …), so TypeScript complained *string is not assignable*.

They tried:

- Renaming the file to `.tsx` (does not fix a type error).
- Hand-writing an `IToastConfig` interface (theme `light | dark | colored`, booleans for the flags, `number | undefined` for `progress`, `React.ReactNode` for `transition` — and `Bounce` did not match `ReactNode`).
- Looking up `@types/react-toastify` — **deprecated**.

What worked: Toastify already exports `ToastOptions`. Annotate the config as `ToastOptions` and the file can stay `.ts`.

> **Exam:** Mount `<ToastContainer />` once in `App`. Call `toast.success` / `toast.error` (or the util wrappers) after the API. Keep toast helpers in `src/utils` under a **generic** name so a library swap touches one file.

> **Pitfall:** Repeating `toast.success( ... , { position, theme, transition })` in every component duplicates config and couples you to React Toastify. Also: `position` is not a free `string`; type the config as `ToastOptions`.

Controlled inputs and clearing the form after success

After a successful insert the **fields still showed the old text**. State was never reset, so the picture of the last product stayed on screen.

Two steps, both required:

1. Make every field **controlled**: `value={formData.name}` (and the same for `description`, `price`, `qty`). The input shows whatever is in state.
2. After a successful POST (and the success toast), `setFormData` **back to the initial blanks**. React re-renders; the inputs go empty.

```
<TextField
  fullWidth
  label="Name of the product"
  name="name"
  variant="outlined"
  value={formData.name}
  onChange={handleChange}
/>
```

```
setFormData({
  name: "",
  description: "",
  price: "",
  qty: "",
});
```

First attempt in class (**25-watt charger** / **Type-C to C charger** / **781** / **5**): they added `value={ ... }` but **forgot the reset**, so the fields stayed filled — and that was not a bug in `value`, it was a missing `setFormData`. Second attempt (**headphone** / **Jabra headphone** / **with mic** / **4000** / **5**): success toast **and** the form cleared. Opening the product list then showed every product they had inserted.

> **Pitfall:** `value={formData.x}` without a post-success reset does **not** clear the form. Controlled inputs only mirror state. Reset the state.

Delete Dialog

Problem: the list can show products but cannot remove one safely

With create working, the list is filling up. Each row now needs **Edit** and **Delete**. Edit is **evening**. Delete is this session — and it must not fire on an accidental click.

Options column and icon buttons

In `ProductsList`, add a heading cell **Options**, and one extra `TableCell` **inside the** `map` so every data row gets the actions.

`@mui/icons-material` was already installed. Class used `IconButton` + `Delete` + `Edit`:

Button	Icon	`color`	Why that colour
Delete	`Delete` (Delete icon)	`error`	Predefined red
Edit	`Edit`	`success`	Green / success theme (not red)

Any other colour is `sx` with a hex or `rgb( … )`. Gap between the two buttons: `sx={{ mr: 2 }}` (about 2 spacing units of margin-right) on the first button.

```
<TableCell>
  <IconButton
    color="error"
    sx={{ mr: 2 }}
    onClick={() => {
      setSelectedProductId(product.id);
      setOpenDeleteDialog(true);
    }}
  >
    <DeleteIcon />
  </IconButton>
  <IconButton color="success">
    <EditIcon />
  </IconButton>
</TableCell>
```

Why a dialog exists

Lecturer: *it should not happen that the user clicked and you deleted. It might have been an accidental click.* Flow:

```
User clicks Delete icon
            |
            v
     Dialog opens
     title: Confirmation
            |
       +----+-----+
       |          |
      Yes         No / backdrop
       |          |
       v          v
 DELETE /product/id   close only
 then refresh list    (row stays)
```

A dialog **never shows by itself**. Like the Drawer from an earlier session, visibility is the boolean `open` prop, driven by state.

Dialog anatomy (from the MUI example they pasted)

Piece	Role in class
`Dialog`	Shell; `open={openDeleteDialog}` ; later `fullWidth`
`DialogTitle`	**Confirmation**
`DialogContent` + `DialogContentText`	Body: **Are you sure you want to delete** (they also mentioned "once deleted you cannot recover it" as a possible line)
`DialogActions`	The two buttons

Button	`variant`	`color`	Click
Yes	`contained` (they first tried `outlined` , then switched)	`success`	run `deleteProduct`
No	`contained`	`error`	`setOpenDeleteDialog(false)`

`onClose` on `Dialog` (backdrop / Escape) does the same as No: `setOpenDeleteDialog(false)` . They skipped `autoFocus` on a default button for now.

MUI also offers sizes, full-screen, slides, and even a form inside a dialog. Class used `fullWidth` and left further customisation as "read the docs".

This state belongs in `useProductsList`, not in the JSX file

Same rule as the rest of the list: **logic lives in the hook**. Class added:

```
const [openDeleteDialog, setOpenDeleteDialog] = useState(false);
```

...and **returned** both values from `useProductsList` so `ProductsList` can bind `open` and the click handlers. Initial value `false` — the dialog is closed until Delete is pressed.

The DELETE URL needs an id: `selectedProductId`

`DELETE /product/:id` cannot run from "the user clicked some trash icon". You must know **which row**.

Each mapped row already has `product.id`. On Delete-icon click, class set **two** states:

1. `setOpenDeleteDialog(true)` — show the dialog.
2. `setSelectedProductId(product.id)` — remember that row.

```
const [selectedProductId, setSelectedProductId] = useState(0);
```

`0` is the unused initial number. When Yes is clicked, `deleteProduct` reads **state**, not the click event:

```
async function deleteProduct() {
  try {
    const response = await apiClient.delete(`/product/${selectedProductId}`);
    console.log(response.data);
    setOpenDeleteDialog(false);
    // then refresh the list (see below)
  } catch (error) {
    console.log(error);
    showErrorToast("Deletion failed");
  }
}
```

Template literals, not string-concat, for the URL. `delete` returns a Promise → `async` / `await`.

Student question: "can we take the id as a prop?"

Nilesh Desale: when we write the function for the delete API call, can we take the id through a prop?

Lecturer: you *can* pass the id into the function — but that is an **argument**, not a prop. **Props belong to components.** A function parameter is just a parameter.

So two legal designs, both stated:

Design	How the id arrives
State (what class shipped)	Click stores `product.id` in `selectedProductId`; `deleteProduct()` reads it
Argument	`deleteProduct(id)` / `onClick={() ⇒ deleteProduct(product.id)}`

Either works. Do not call a function argument a prop.

The list on screen is stale until you update it

First live delete (**keyboard**): they clicked Delete, saw a `SELECT` for that row in the backend logs, clicked Yes, and **MySQL no longer had the row** — but the table on screen still showed Keyboard. Two leftovers:

1. The dialog is still open.
2. The React list state still holds the deleted product.

Both must be fixed **after** a successful `DELETE`.

Two ways to refresh the frontend (both taught):

Strategy	How
Refetch (what they shipped)	Close the dialog, then call the same fetch-all you use on mount. The new array will not contain the deleted id.
Local filter	`setProducts(products.filter((p) ⇒ p.id ≠= selectedProductId))` — drop that id from the array you already have.

```
Yes
 |
 v
DELETE /product/3     ⟶  row gone in MySQL
 |
 v
setOpenDeleteDialog(false)
 |
 v
fetch products again  ⟶  setProducts(newArray)
 |
 v
table re-renders without that row
```

Demos after the refetch was added: **headphone** Yes → gone from the table; **SSD** Yes → gone. No → dialog disappears and the row stays.

`useCallback` around `deleteProduct` was left as "you have seen enough examples; wrap functions yourself."

> **Exam:** Never delete on the icon click. Open a `Dialog` (`open` + state). Capture `product.id` in state (or pass it as a **function argument**). `apiClient.delete(\`/product/${id}\`)`. Then close the dialog and **either refetch or filter** the local list — otherwise the database and the table disagree.

> **Pitfall:** A successful DELETE + a table that still shows the row is not an API bug. You forgot to update React state.

Homework, Validation and What Comes Next

Homework (CreateProduct)

Move **all of the logical part** — `formData` state, `handleChange`, `handleSubmit` — into a hook named `useCreateProduct`. The component only calls the hook and uses what it returns.

Wrap `handleChange` and `handleSubmit` in `useCallback` with a **correct dependency array**. Those functions close over `formData`, so `formData` belongs in the deps.

This is the same extract-a-hook move they have already done for the list.

Validation (not today)

Om Kamble: price and quantity must be validated — a user can type a negative.

Lecturer: yes. The whole form will be validated with **React Hook Form**: required fields, quantity rules, price rules. Not in this episode.

Still coming

When	What
Evening of this day	**Update** (edit icon → load product → PUT)
After that	Form validation (React Hook Form)
After that	Basic authentication, then **role-based** authentication so admin and customer see different menus

Admin asked students to share **2 November evening** notes and files; that is logistics, not study content.

Combined Worked Example

A Vite + React + TypeScript app with MUI, Axios and React Toastify already installed. These files are the day's work in one place: typed form, 12-column grid, `apiClient.post`, toast utils, controlled reset, list options column, confirmation dialog, `DELETE`, refetch.

Assumed packages: `@mui/material`, `@mui/icons-material`, `@emotion/react`, `@emotion/styled`, `axios`, `react-toastify`, `react-router-dom`. Express from Day 8 is listening with `POST /product`, `GET /product`, `DELETE /product/:id`.

```
npm i @mui/material @mui/icons-material @emotion/react @emotion/styled axios react-toastify react-router-dom
```

`src/types.ts`

```ts
export interface IFormData {
  name: string;
  description: string;
  price: string;
  qty: string;
}

export interface IProduct {
  id: number;
  name: string;
  description: string;
  price: number | string;
  quantity: number | string;
  qty?: number | string;
}
```

`src/api/apiClient.ts`

```ts
import axios from "axios";

export const apiClient = axios.create({
  baseURL: "http://localhost:3000",
  timeout: 5000,
});
```

src/utils/toastUtil.ts

```ts
import { toast, Bounce, type ToastOptions } from "react-toastify";

const toastConfig: ToastOptions = {
  position: "top-right",
  autoClose: 5000,
  hideProgressBar: false,
  closeOnClick: true,
  pauseOnHover: true,
  draggable: true,
  theme: "colored",
  transition: Bounce,
};

export function showSuccessToast(message: string) {
  toast.success(message, toastConfig);
}

export function showErrorToast(message: string) {
  toast.error(message, toastConfig);
}
```

src/hooks/useCreateProduct.ts

Homework extract: state + callbacks live here, not in the JSX file.

```ts
import { useCallback, useState } from "react";
import type { ChangeEvent, FormEvent } from "react";
import { apiClient } from "../api/apiClient";
import { showErrorToast, showSuccessToast } from "../utils/toastUtil";
import type { IFormData } from "../types";

const emptyForm: IFormData = {
  name: "",
  description: "",
  price: "",
  qty: "",
};

export function useCreateProduct() {
  const [formData, setFormData] = useState<IFormData>(emptyForm);

  const handleChange = useCallback(
    (event: ChangeEvent<HTMLInputElement>) => {
      setFormData({
        ...formData,
        [event.target.name]: event.target.value,
      });
```

```
    },
    [formData]
  );

  const handleSubmit = useCallback(
    async (event: FormEvent<HTMLFormElement>) => {
      event.preventDefault();
      try {
        const response = await apiClient.post("/product", formData);
        console.log(response.data);
        showSuccessToast("Product created");
        setFormData(emptyForm);
      } catch (error) {
        console.log(error);
        showErrorToast("Product creation failed");
      }
    },
    [formData]
  );

  return { formData, handleChange, handleSubmit };
}
```

src/pages/CreateProduct.tsx

```
import { Alert, Box, Button, Grid, TextField } from "@mui/material";
import { useCreateProduct } from "../hooks/useCreateProduct";

export function CreateProduct() {
  const { formData, handleChange, handleSubmit } = useCreateProduct();

  return (
    <Box sx={{ mt: "30px" }}>
      <Alert severity="info">Add a New Product</Alert>

      <Box component="form" onSubmit={handleSubmit} sx={{ mt: 2 }}>
        <Grid container spacing={5} sx={{ mb: 4 }}>
          <Grid size={6}>
            <TextField
              fullWidth
              label="Name of the product"
              name="name"
              variant="outlined"
              value={formData.name}
              onChange={handleChange}
            />
          </Grid>
          <Grid size={6}>
            <TextField
```

```
              fullWidth
              label="Description"
              name="description"
              variant="outlined"
              value={formData.description}
              onChange={handleChange}
            />
          </Grid>
        </Grid>

        <Grid container spacing={5}>
          <Grid size={6}>
            <TextField
              fullWidth
              label="Price"
              name="price"
              variant="outlined"
              value={formData.price}
              onChange={handleChange}
            />
          </Grid>
          <Grid size={6}>
            <TextField
              fullWidth
              label="Quantity"
              name="qty"
              variant="outlined"
              value={formData.qty}
              onChange={handleChange}
            />
          </Grid>
        </Grid>

        <Box sx={{ mt: 3 }}>
          <Button type="submit" variant="contained">
            Add Product
          </Button>
        </Box>
      </Box>
    </Box>
  );
}
```

src/hooks/useProductsList.ts

```
import { useCallback, useEffect, useState } from "react";
import { apiClient } from "../api/apiClient";
import { showErrorToast, showSuccessToast } from "../utils/toastUtil";
import type { IProduct } from "../types";
```

```
export function useProductsList() {
  const [products, setProducts] = useState<IProduct[]>([]);
  const [openDeleteDialog, setOpenDeleteDialog] = useState(false);
  const [selectedProductId, setSelectedProductId] = useState(0);

  const fetchProducts = useCallback(async () => {
    try {
      const response = await apiClient.get("/product");
      setProducts(response.data);
    } catch (error) {
      console.log(error);
      showErrorToast("Could not load products");
    }
  }, []);

  useEffect(() => {
    fetchProducts();
  }, [fetchProducts]);

  const deleteProduct = useCallback(async () => {
    try {
      const response = await apiClient.delete(`/product/${selectedProductId}`);
      console.log(response.data);
      setOpenDeleteDialog(false);
      showSuccessToast("Product deleted");
      await fetchProducts();
    } catch (error) {
      console.log(error);
      showErrorToast("Deletion failed");
    }
  }, [selectedProductId, fetchProducts]);

  return {
    products,
    openDeleteDialog,
    setOpenDeleteDialog,
    selectedProductId,
    setSelectedProductId,
    deleteProduct,
  };
}
```

src/pages/ProductsList.tsx

```
import DeleteIcon from "@mui/icons-material/Delete";
import EditIcon from "@mui/icons-material/Edit";
import {
  Button,
```

```
  Dialog,
  DialogActions,
  DialogContent,
  DialogContentText,
  DialogTitle,
  IconButton,
  Table,
  TableBody,
  TableCell,
  TableHead,
  TableRow,
} from "@mui/material";
import { useProductsList } from "../hooks/useProductsList";

export function ProductsList() {
  const {
    products,
    openDeleteDialog,
    setOpenDeleteDialog,
    setSelectedProductId,
    deleteProduct,
  } = useProductsList();

  return (
    <>
      <Table>
        <TableHead>
          <TableRow>
            <TableCell>ID</TableCell>
            <TableCell>Name</TableCell>
            <TableCell>Description</TableCell>
            <TableCell>Price</TableCell>
            <TableCell>Quantity</TableCell>
            <TableCell>Options</TableCell>
          </TableRow>
        </TableHead>
        <TableBody>
          {products.map((product) => (
            <TableRow key={product.id}>
              <TableCell>{product.id}</TableCell>
              <TableCell>{product.name}</TableCell>
              <TableCell>{product.description}</TableCell>
              <TableCell>{product.price}</TableCell>
              <TableCell>{product.quantity ?? product.qty}</TableCell>
              <TableCell>
                <IconButton
                  color="error"
                  sx={{ mr: 2 }}
                  onClick={() => {
                    setSelectedProductId(product.id);
                    setOpenDeleteDialog(true);
```

```
                  }}
                >
                  <DeleteIcon />
                </IconButton>
                <IconButton color="success">
                  <EditIcon />
                </IconButton>
              </TableCell>
            </TableRow>
          ))}
        </TableBody>
      </Table>

      <Dialog
        open={openDeleteDialog}
        onClose={() => setOpenDeleteDialog(false)}
        fullWidth
      >
        <DialogTitle>Confirmation</DialogTitle>
        <DialogContent>
          <DialogContentText>
            Are you sure you want to delete?
          </DialogContentText>
        </DialogContent>
        <DialogActions>
          <Button variant="contained" color="success" onClick={deleteProduct}>
            Yes
          </Button>
          <Button
            variant="contained"
            color="error"
            onClick={() => setOpenDeleteDialog(false)}
          >
            No
          </Button>
        </DialogActions>
      </Dialog>
    </>
  );
}
```

src/App.tsx

```
import { ToastContainer } from "react-toastify";
import "react-toastify/dist/ReactToastify.css";
import { BrowserRouter, Link, Route, Routes } from "react-router-dom";
import { CreateProduct } from "./pages/CreateProduct";
import { ProductsList } from "./pages/ProductsList";
```

```
export default function App() {
  return (
    <BrowserRouter>
      <ToastContainer />
      <nav>
        <Link to="/">Products</Link>
        {" | "}
        <Link to="/create">Add a New Product</Link>
      </nav>
      <Routes>
        <Route path="/" element={<ProductsList />} />
        <Route path="/create" element={<CreateProduct />} />
      </Routes>
    </BrowserRouter>
  );
}
```

End-to-end trace

1. `App` mounts. `<ToastContainer />` is on every page. `/create` renders `CreateProduct`. `useCreateProduct` starts with four empty strings.
2. The user types `mouse` in Name. `onChange` fires; `event.target.name` is `"name"`; `setFormData` spreads the old object and writes `name: "mouse"`. The `value={formData.name}` binding shows `mouse`.
3. Description `wireless mouse`, price `654`, qty `13` land in the same object via the same handler because each input's `name` matches a key.
4. **Add Product** submits the `Box component="form"`. `handleSubmit` runs `preventDefault()`, then `apiClient.post("/product", formData)`. Axios hits `baseURL + "/product"` with that JSON.
5. Express inserts a row and replies `{ message: "product inserted", insertedId: 4 }`. The try path calls `showSuccessToast("Product created")` — Bounce, colored, top-right — and `setFormData(emptyForm)`. The four fields go blank.
6. If the SQL table name is wrong, Express returns 500, Axios throws, `catch` calls `showErrorToast("Product creation failed")`. The page stays up.
7. On `/`, `useProductsList` fetches `GET /product` and fills the table, including an **Options** cell with a red delete icon and a green edit icon.
8. Delete on Keyboard stores `selectedProductId =` that row's id and sets `openDeleteDialog` to `true`. The dialog title is **Confirmation**.
9. **No** (or backdrop) only sets `open` back to `false`. Keyboard stays.
10. **Yes** runs `apiClient.delete("/product/" + selectedProductId)`, closes the dialog, and refetches. Keyboard disappears from MySQL **and** from the table. The local-filter alternative

(`filter` out that id) would skip the extra GET.

Quick Revision Checklist

Topic	One-line takeaway
`TextField`	MUI input; `label` , `name` , `variant` (`outlined` / `filled` / `standard`)
Form wrapper	Raw `<form>` **or** `<Box component="form">` to keep MUI `Box` behaviour
`Stack`	Children one after another; `spacing` is the vertical gap
Hardcoded `width: 300`	Same 300 px on phone, tablet and laptop — not responsive
Grid	Viewport = 12 columns; a row should sum to 12 (`6+6` , `4+4+4` , `8+4` , `2+8+2`)
`container`	This Grid is the **row**
`size={6}`	This Grid is a **column** taking 6 of 12
Wrap on small screens	Side-by-side sections drop onto a new line so the UI does not break
`fullWidth`	Field fills its **column**, not the whole browser
`spacing`	Gap between cells **inside** one container
Two containers	`rowSpacing` will not gap them; use one parent grid or `sx={{ mb: 4 }}`
Button	`type="submit"` ; `variant` is `text` / `contained` / `outlined` (not "content")
Form state	One `formData` object; keys = `name` attributes (`qty` , not `quantity` , if that is the key)
`IFormData`	Interface in `types.ts` ; price/qty are `number` only if the UI will do arithmetic
`handleChange`	`ChangeEvent<HTMLInputElement>` ; spread + `[event.target.name]`
`handleSubmit`	On the form; `preventDefault()` ; then API
`apiClient.post`	`await apiClient.post("/product", formData)` ; read `response.data`
try/catch	A 500 must not crash the SPA
Toastify	`<ToastContainer />` once in `App` ; `toast.success` / `toast.error` at the call site

Topic	One-line takeaway
`toastUtil`	`src/utils/toastUtil.ts` — generic name; `showSuccessToast` / `showErrorToast`
`ToastOptions`	Type the shared config; `position` is not a free `string`
Controlled + reset	`value={formData.x}` **and** `setFormData(empty)` after success
Homework	`useCreateProduct` + `useCallback` with `formData` in deps
Options column	Extra `TableCell` inside `map`; `IconButton` + Delete (`error`) + Edit (`success`)
Dialog	`open` is state; Yes deletes, No / `onClose` only closes
`selectedProductId`	Remember which row; or pass id as a **function argument**, never call that a prop
After DELETE	Close dialog, then **refetch** or **filter** local state

---

Practice Exercises

Concept

1. Why does the create page use MUI `TextField` instead of a raw HTML `<input>`, and which three `variant` values did the lecture name?
2. What are the two legal ways to get a real `<form>` around the MUI fields, and what extra do you keep if you use `Box`?
3. Why did the lecture abandon `Stack` plus `width: 300`?
4. State the 12-column rule and list the four partitions drawn in class.
5. What is the difference between `<Grid container>` and `<Grid size={6}>`?
6. What does `fullWidth` on a `TextField` fill — the browser, or something else?
7. Why did `rowSpacing` on the two product-form grids fail to separate the rows? What two fixes were discussed?
8. When should `price` and `qty` be `number` in `IFormData`, and when may they stay `string`?
9. Why must every input's `name` match a key of `formData`?
10. What job does `event.preventDefault()` do on submit, and what goes wrong without it?
11. Why call `apiClient.post("/product", formData)` instead of `axios.post("http://localhost:3000/product", formData)`?

12. After a 200 insert, which object is "the response the API used to send" — `response` or `response.data`?
13. Why put `<ToastContainer />` in `App` and not inside `CreateProduct`?
14. Give the two reasons the lecture moved toast calls into `src/utils/toastUtil.ts`, and why that file is not named `reactToastify.ts`.
15. Why did TypeScript reject `position: "top-right"` on a plain config object, and which type fixed it?
16. Why is `value={formData.name}` alone not enough to clear the form after insert?
17. Why must Delete open a dialog instead of calling the API on the icon click?
18. Nilesh asked whether the delete function can take the id as a "prop". What was the correction?
19. After `DELETE` succeeds, the row is gone in MySQL but still visible. What two frontend repairs did the lecture name?

Application

20. Write a `TextField` for quantity that will work with the generic `handleChange` from class (`name` must be `qty`).
21. Write a `6 + 6` Grid row for `name` and `description`, both `fullWidth`, container `spacing={5}`.
22. Write `handleChange` typed with `ChangeEvent<HTMLInputElement>`.
23. Write the `try/catch` body of `handleSubmit` after `preventDefault`: POST, success toast, reset, error toast.
24. Write `showSuccessToast` / `showErrorToast` plus a shared `ToastOptions` config (top-right, colored, Bounce).
25. Write the Delete-icon `onClick` that both opens the dialog and stores `product.id`.
26. Write `deleteProduct` so it calls `DELETE /product/:id`, closes the dialog, and refetches.

Predict the output

27. `formData` is `{ name: "SSD", description: "Seagate", price: "4567", qty: "12" }`. The name-field handler is `setFormData({ name: event.target.value })` and the user types a space. What is state afterwards?
28. Two `Grid size={6}` fields have no `fullWidth`. What do you see inside each half-row?
29. `rowSpacing={5}` is set on **each** of two sibling `Grid container`s. Is there a gap between the two rows?
30. The quantity input is `name="quantity"` but state has a `qty` key. The user types `18`. What does `formData.qty` stay as, and what extra key appears?

31. `<ToastContainer />` was forgotten in `App`. `showSuccessToast("Product created")` runs after a 201. What does the user see?
32. Delete icon sets only `openDeleteDialog` to `true`. `selectedProductId` is still `0`. The user clicks Yes. Which URL is requested?
33. `deleteProduct` awaits the DELETE and does nothing else. MySQL lost the row. What does the table still show, and is the dialog still open?

Spot the bug

34. `<Box onSubmit={handleSubmit}>` without `component="form"`, and a `<Button type="submit">`. Why might submit never fire?
35. `<Grid size={6}>` used as the outer wrapper with two more `<Grid size={6}>` inside, and **no** `container` on the parent. What layout rule was skipped?
36. `toast.success("Product created", { position: "top-right", ... })` copied into ten pages. Tomorrow Toastify is replaced. What extra cost did the lecture warn about?
37. `useCreateProduct` returns handlers that are plain functions (no `useCallback`), and the homework asked for `useCallback` with proper deps. Which dependency must be listed, and why?
38. Yes button is `onClick={deleteProduct()}` (with parentheses). What happens on render?
39. After DELETE they run `setOpenDeleteDialog(false)` but neither refetch nor filter. Name the leftover bug.

Practice Answers

1. `TextField` is the MUI-styled input that follows the library's look (label, variants, `fullWidth`) instead of a bare browser box. Variants named in class: `outlined`, `filled`, `standard`.

2. A native `<form>`, or `<Box component="form">`. The Box path still renders a form element but keeps MUI `Box` features (`sx`, spacing, layout).

3. `300` px is forced on every device. The form does not share the 12-column grid or wrap cleanly; it is a fixed strip, not a responsive layout.

4. The library splits the width into 12 equal columns; sections in one row should add to 12. Drawn: `6+6`, `4+4+4`, `8+4`, `2+8+2`.

5. `container` marks the Grid as the **row** that owns the 12-column track. `size={6}` without `container` is a **column** that occupies six of those columns.

6. It fills the **parent column** (the `size={6}` cell), not the whole browser.

7. Those two grids are separate containers, so `rowSpacing` only spaces children *inside* each one. Fixes: put both rows in one parent `Grid container` (Shivam Soni), or `sx={{ mb: 4 }}` on the first row (what they shipped).

8. `number` if the client will do arithmetic; `string` if the value is only collected and sent to the server. `event.target.value` is a string either way.

9. `[event.target.name]` writes whatever the attribute says. A mismatch creates a new key and leaves the real state field empty.

10. It stops the browser's default form navigation / reload. Without it the page refreshes, React state is lost, and the async POST is interrupted from the user's point of view.

11. `apiClient` is the existing Axios instance that already has `baseURL` and `timeout`. You only pass the sub-path `/product` and the body.

12. `response.data`. `response` is the whole Axios object (status, headers, data, timing).

13. The container is a single app-wide mount point. Any later `toast.success` / `showSuccessToast` renders into that container. Putting one in every page is the opposite of the util discussion.

14. (1) Do not repeat the same config in 50–100 components. (2) If the library dies, only the util file changes. The name stays generic (`toastUtil`) so components never mention React Toastify.

15. TypeScript widened `"top-right"` to `string`, but Toastify only accepts a fixed position union. Annotate the object as `ToastOptions`.

16. Controlled inputs **display** state. They clear only when you `setFormData` back to blanks after success.

17. The click might be accidental. Confirm first; call DELETE only on Yes.

18. You may pass the id into the function — that is an **argument**. Props are for **components**.

19. Close the dialog, then either **refetch** `GET /product` or **filter** the deleted id out of the local array.

20.

```
<TextField
  fullWidth
  label="Quantity"
  name="qty"
  variant="outlined"
  value={formData.qty}
  onChange={handleChange}
/>
```

21.

```
<Grid container spacing={5}>
  <Grid size={6}>
    <TextField fullWidth label="Name of the product" name="name" variant="outlined" />
  </Grid>
  <Grid size={6}>
    <TextField fullWidth label="Description" name="description" variant="outlined" />
  </Grid>
</Grid>
```

22.

```
function handleChange(event: ChangeEvent<HTMLInputElement>) {
  setFormData({
    ...formData,
    [event.target.name]: event.target.value,
  });
}
```

23.

```
try {
  const response = await apiClient.post("/product", formData);
  console.log(response.data);
  showSuccessToast("Product created");
  setFormData({ name: "", description: "", price: "", qty: "" });
} catch (error) {
  console.log(error);
  showErrorToast("Product creation failed");
}
```

24.

```
import { toast, Bounce, type ToastOptions } from "react-toastify";

const toastConfig: ToastOptions = {
  position: "top-right",
  autoClose: 5000,
  hideProgressBar: false,
  closeOnClick: true,
  pauseOnHover: true,
  draggable: true,
  theme: "colored",
  transition: Bounce,
};
```

```
export function showSuccessToast(message: string) {
  toast.success(message, toastConfig);
}

export function showErrorToast(message: string) {
  toast.error(message, toastConfig);
}
```

25.

```
onClick={() => {
  setSelectedProductId(product.id);
  setOpenDeleteDialog(true);
}}
```

26.

```
async function deleteProduct() {
  try {
    const response = await apiClient.delete(`/product/${selectedProductId}`);
    console.log(response.data);
    setOpenDeleteDialog(false);
    await fetchProducts();
  } catch (error) {
    console.log(error);
    showErrorToast("Deletion failed");
  }
}
```

27. Something like `{ name: "SSD " }` only. `description`, `price` and `qty` are gone because the object was replaced, not spread.

28. Two short default-width inputs sitting in the left side of each half. The columns are wide; the fields are not.

29. No. Each container's `rowSpacing` does not apply between sibling containers.

30. `formData.qty` stays `""` (or whatever it was). A new `quantity: "18"` key is added.

31. Nothing on screen. The toast function runs but there is no container to host it. The insert may still have succeeded.

32. `DELETE /product/0` (or `/product/0` on the configured host). The real row id was never stored.

33. The table still lists that product, and the dialog stays open.

34. `Box` defaults to a `div`. A submit button does not submit a `div`. Use `component="form"` or a real `<form>`.

35. Without `container`, there is no row managing the 12-column track. Children will not lay out as `6 + 6` columns.

36. Every page imported and configured React Toastify. Replacing the library means editing all ten pages ("Lanka") instead of one util file.

37. `formData`, because both handlers read the current object (spread on change; POST body and reset on submit). An empty `[]` would freeze them on the first empty object.

38. `deleteProduct` runs immediately when the dialog renders (including the first time `open` becomes true), not when Yes is clicked. Use `onClick={deleteProduct}`.

39. The dialog is closed but the deleted product **remains in the table** until a refetch or a local filter updates React state.